**Problem Statement**

In computer science, a priority queue is an abstract data type which is like a regular queue, but where additionally each element has a "priority" associated with it. In a priority queue, an element with high priority is served before an element with low priority. - Wikipedia

---

In this problem we will test your knowledge on Java Priority Queue.

There are a number of students in a school who wait to be served. Two types of events, *ENTER* and *SERVED*, can take place which are described below.

- *ENTER*: A student with some priority enters the queue to be served.
- *SERVED*: The student with the highest priority is served (removed) from the queue.

A unique id is assigned to each student entering the queue. The queue serves the students based on the following criteria (priority criteria):

1. The student having the highest *Cumulative Grade Point Average* (CGPA) is served first.
2. Any students having the *same CGPA* will be served by name in ascending case-sensitive alphabetical order.
3. Any students having the *same CGPA and name* will be served in ascending order of the id.

Create the following two classes:

- The *Student* class should implement:
    - The constructor Student(int id, String name, double cgpa).
    - The method int getID() to return the id of the student.
    - The method String getName() to return the name of the student.
    - The method double getCGPA() to return the CGPA of the student.
- The *Priorities* class should implement the method List<Student> getStudents(List<String> events) to process all the given events and return all the students yet to be served in the priority order.

Input Format

The first line contains an integer, , describing the total number of events. Each of the subsequent lines will be of the following two forms:

- ENTER name CGPA id: The student to be inserted into the priority queue.
- SERVED: The highest priority student in the queue was served.

The locked stub code in the editor reads the input and tests the correctness of the *Student* and *Priorities* classes implementation.

Constraints

- 
- 
- 
- 

Output Format

The locked stub code prints the names of the students yet to be served in the priority order. If there are no such student, then the code prints EMPTY.

**Program:**

```
import java.util.*;


// Student class
class Student {
  private int id;
  private String name;
  private double cgpa;

  public Student(int id, String name, double cgpa) {
    this.id = id;
    this.name = name;
    this.cgpa = cgpa;
  }

  public int getID() {
    return id;
  }

  public String getName() {
    return name;
  }

  public double getCGPA() {
```

```
        return cgpa;
    }
}

// Priorities class
class Priorities {
    public List<Student> getStudents(List<String> events) {
        // Priority Queue with custom comparator
        PriorityQueue<Student> pq = new PriorityQueue<>(
            (s1, s2) -> {
                if (Double.compare(s2.getCGPA(), s1.getCGPA()) != 0) {
                    return Double.compare(s2.getCGPA(), s1.getCGPA()); // Higher CGPA first
                } else if (!s1.getName().equals(s2.getName())) {
                    return s1.getName().compareTo(s2.getName()); // Alphabetical order
                } else {
                    return Integer.compare(s1.getID(), s2.getID()); // Lower ID first
                }
            }
        );

        for (String event : events) {
            String[] parts = event.split(" ");
            if (parts[0].equals("ENTER")) {
                String name = parts[1];
                double cgpa = Double.parseDouble(parts[2]);
                int id = Integer.parseInt(parts[3]);
                pq.add(new Student(id, name, cgpa));
            } else if (parts[0].equals("SERVED")) {
                pq.poll(); // remove highest priority student
            }
        }
```

```
        List<Student> remaining = new ArrayList<>();
        while (!pq.isEmpty()) {
            remaining.add(pq.poll());
        }
        return remaining;
    }
}

// Solution class (main driver)
public class Solution {
    private final static Scanner scan = new Scanner(System.in);
    private final static Priorities priorities = new Priorities();

    public static void main(String[] args) {
        int totalEvents = Integer.parseInt(scan.nextLine());
        List<String> events = new ArrayList<>();

        while (totalEvents-- != 0) {
            String event = scan.nextLine();
            events.add(event);
        }

        List<Student> students = priorities.getStudents(events);

        if (students.isEmpty()) {
            System.out.println("EMPTY");
        } else {
            for (Student st : students) {
                System.out.println(st.getName());
            }
```

```
    }
  }
}
```

**Sample Input 0**

```
12
ENTER John 3.75 50
ENTER Mark 3.8 24
ENTER Shafaet 3.7 35
SERVED
SERVED
ENTER Samiha 3.85 36
SERVED
ENTER Ashley 3.9 42
ENTER Maria 3.6 46
ENTER Anik 3.95 49
ENTER Dan 3.95 50
SERVED
```

**Sample Output 0**

```
Dan
Ashley
Shafaet
Maria
```